CMSC 218 Review

Pandas

- Group by : Collapses rows that share the same category into one row, can perform an aggregate operation on the collapsed rows

  - Ex. `df.groupby(["col1"]).mean()`

  - To group across multiple columns, use agg():
    `df.agg(col1mean = ("col1", "mean"), col2median = ("col2", "median"))`

- merge : merge by a common column
  - Ex. `merged = pd.merge(df1, df2, on = "x", how = "inner")`
  - how :
    - inner – merge takes intersection of both `df1["x"]`, `df2["x"]`
    - outer – merge takes union of `df1["x"]`, `df2["x"]`
    - left – merge uses `df1["x"]` as index
    - right – merge uses `df2["x"]` as index
  - outer, left, right can produce NaN.

Sampling

- 68-95-99.7 rule – 68% of samples fall within 1 SD,
  95% of samples fall within 2 SD
  99.7% of samples fall within 3 SD

- Stratified sampling – point estimate is a weighted average of sub populations
  - $\bar{x} = w_1 \bar{x}_1 + \dots + w_n \bar{x}_n$
  - $Var(\bar{x}) = w_1^2 Var(\bar{x}_1) + \dots w_n^2 Var(\bar{x}_n)$

- Ensures representativeness of all population according to prior weights

- 95% CI for proportions:

  - $CI = \hat{p} \pm 1.96 \cdot SE(\hat{p})$

  $\leq \hat{p} \pm 1.96\sqrt{\frac{0.5(0.5)}{n}}$

Linear model:

- $Y = mX + b + \varepsilon$

- $E[Y] = mE[X] + b$
- $Var(Y) = m^2 Var(X) + Var(\varepsilon)$
  - Signal-to-noise ratio:

$$SNR = \frac{m^2 Var(X)}{Var(\varepsilon)}$$

- $Cov(X,Y) = E[(X - E[X])(Y - E[Y])]$

  Affected by scaling: $Cov(cX, Y) = c\,Cov(X,Y)$

  - Measures the scale of association between X and Y

- Correlation $\rho = \frac{Cov(X,Y)}{\sqrt{Var(X)\,Var(Y)}}$

  - Not affected by scaling X or Y
  - Measures the strength

- 1-D OLS: $\begin{cases} \hat{m} = \frac{Cov(X,Y)}{Var(X)} \\ \hat{b} = E[Y] - \hat{m}E[X] \end{cases}$ $\Rightarrow$ if X, Y standardized, $\hat{m} = \frac{Cov(X,Y)}{Var(X)} = Cov(X,Y)$

  $\rho = \frac{Cov(X,Y)}{\sqrt{Var(X)Var(Y)}} = Cov(X,Y)$

  $\Rightarrow \hat{m} = \rho$.

- Bias variance decomposition:

$$E[(\hat{y} - y)^2] = \left(E[\hat{m}] - m\right)^2 + \left(E[\hat{s}] - s\right)^2 + Var(\hat{m}) + Var(\hat{s}) + Var(\varepsilon)$$

$Bias^2(\hat{m})$ $Bias^2(\hat{s})$ Variance Error

- Multivariate OLS:
  - $y = X\beta + \varepsilon$, $X \in \mathbb{R}^{n \times s}$, $\beta \in \mathbb{R}^{s \times p}$, $\varepsilon \in \mathbb{R}^{n \times p}$, $y \in \mathbb{R}^{n \times p}$
  - $\underset{\beta}{\operatorname{argmin}} |Y - X\beta|_2^2 \Rightarrow \hat{\beta} = (X^T X)^{-1} X^T y \Rightarrow \hat{y} = X\hat{\beta} = X(X^T X)^{-1} X^T y$
  - Bias $\neq \frac{Cov(X_i, y_i)}{Var(X_i)}$, so $\hat{m} \neq p$ after standardizing X, y.
  - A large R^2 only tells that our predictor is important in context of other predictors.

- $R^2 = 1 - \frac{\sum (\hat{y} - y)^2}{\sum (\hat{y} - \bar{y})^2} = 1 - \frac{SSE}{SST}$

- Measures how much better we do than the baseline model, $\bar{y}$.

- Ridge regression - smooth predictions by adding L2 regularizer:

$$\underset{\beta}{\operatorname{argmin}} |Y - X\beta|_2^2 + \alpha |\beta|_2^2 \Rightarrow \hat{\beta} = (X^T X - \lambda I)^{-1} Y \Rightarrow \hat{y} = (X^T X - \lambda I)^{-1} y$$

Building a Regression Predictor

- Induction model: Given (x, y) pairs, produce $f(\theta)$.
- Prediction model: Given x, y as inputs, organizes the inputs in (x, y) pairs

Typical regression pipeline:

$X = f[\ldots]$

$y = f[\ldots]$

X = StandardScaler().fit_transform(X)

$X_{train}, X_{test}, y_{train}, y_{test}$ = train_test_split(X, y)

model = LinearRegression()

model.fit(X_{train}, y_{train})

y_{pred} = model.predict(y_{train})

r2_score(y_{test}, y_{pred})

Generalization

- T_e - expected testing loss
- T_r - expected training loss
- $T_e - T_r$ = G (generalization gap)

$(T_e - T_r) + T_r = T_e$

G

- A complex model is well-fit to the training data, and we reach it is less generalizable — G ↑, T_r ↓.
- A simple model likely can be well generalized, but has bad training error — G ↓, T_r ↑.
- A good model jointly minimizes G, T_r.

This tradeoff is also seen in the Bias-Variance Decomposition for squared loss:

$E[\ell(y_{pred}, y_{test})] = E[\ell(\hat{f}(x), y_{true})]$

$E[(y_{test} - \hat{f}(x))^2] = (E[\hat{f}(x)] - y_{true})^2 + Var(\hat{f}(x))^2 + \sigma^2$

Bias Variance Noise

- A simple model has less variance, but more bias
- A complex model has low bias, high variance.
- OLS jointly optimizes the bias and variance by minimizing squared loss.

Autoregression

- Autoregression: Using past values of y to predict future values of y.
- Any model that predicts an outcome based on time can may be subject to autoregression.

- AR(1) model: $y_t = \underbrace{\beta_{y_{t-1}} y_{t-1}}_{\text{AR(1) piece}} + \underbrace{X\beta}_{\text{other variables}}$

Typical AR(1) pipeline:

```
df['y_t-1'] = df['y'].shift(1)
df.dropna(how='any')

X = df[...]
y = df[...]

tscv = TimeSeriesSplit()
for train_idx, test_idx in tscv.split(X)
    X_train, X_test = X[train_idx], X[test_idx]
    y_train, y_test = y[train_idx], y[test_idx]

model = LinearRegression
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
r2score(y_test, y_pred)
```

- Longitudinal model — uses $y_{t-k}, \dots y_{t-1}$ to predict y_t
- Cross-sectional model — uses covariates X to predict y_t

Logistic Regression

Given (x_i, y_i), define the probability y_i belongs to class 1 as

$$\hat{p}_i = \frac{1}{1 + e^{-w^T x_i + b}}$$

Then, minimizes cross-entropy loss:

$$\ell(y, \hat{p}) = \sum_{i=1}^{n} y_i \log(\hat{p}_i) + (1 - y_i)(1 - \hat{p}_i)]$$

- By scaling x_i's to $\hat{p}_i$'s, reduces influence of x_i's that are outliers / high leverage points

Classification

- Binary Classification: Train model on weights w, bias b s.t. the decision boundary $w^T x + b$ maximizes correct predictions.
- Confusion Matrix:

Predicted \ Actual	T	N
T	TP	FP
N	FN	TP

Multiclass Classification

- Instead of $y_i \in \{0, 1\}$, now $y_i \in \mathcal{L}$, where $\mathcal{L} = \{1, \dots, k\}$.
- Models for multiclass classification: Logistic Regression, NN/CNN with Cross Entropy loss
- Metrics:
  - Accuracy: $\frac{TP + TN}{TP + FP + TN + FP}$
  - Precision: $\frac{TP}{TP + FP} = \frac{\text{True positives}}{\text{\# of predicted positives}}$
    - How accurate are the model's positive classifications
  - Recall: $\frac{TP}{TP + FN} = \frac{\text{True positives}}{\text{\# of actual positives}}$

- What is the detection rate for true positives?

Ex.

Actual \ Predicted	1	2	3
1	n_{11}	n_{12}	n_{13}
2	n_{21}	n_{22}	n_{23}
3	n_{31}	n_{33}	n_{33}

Precision of class 2: $\dfrac{n_{22}}{n_{12} + n_{22} + n_{32}}$

Recall of class 2: $\dfrac{n_{22}}{n_{21} + n_{22} + n_{23}}$

Two Sample Z-Test

H_0: $m_A = m_B$ There is no difference in the two populations.

H_A: $m_A \neq m_B$ There is a difference in the two populations.

$$z^* = \frac{\bar{x}_A - \bar{x}_B - 0}{\frac{\sigma_A}{\sqrt{n_A}} + \frac{\sigma_B}{\sqrt{n_B}}}$$

If $P(z > z^*) < \alpha$, reject H_0 for H_A.

- P-value is the false discovery rate.

Efficiency vs. Robustness

- Efficiency - tries to use as much of the training data as possible to reach a conclusion
- Robustness - protects against bad assumptions, noise, or inaccurate data
  - Regularization is an example of robustness. Ridge regression, for example imposes prior $w \sim N(0, \frac{1}{\lambda})$.
    - Trade off a small increase in bias for a large decrease in variance
  - Other robustness methods:
    - Acquire more data to decrease leverage of an influential point
    - Convert data to percentiles or ranks (Low Rank Prior)
      - Can lose some of the data's meaningfulness, but improves robustness
    - Find a lower rank representation of data (PCA)

PCA

- Reduce high-dimensional data along a lower-dimensional basis composed of principal components (PCs).
  - Each PC is in the direction of greatest variance in the data
- To find which basis is the result of PCA check:
  - Does the basis vector have larger components in various meaningful axes
  - Does the basis keep position/negative correlations as exhibited in the data?

Neural Networks

Forward Propagation:

- Given input layer $X \in \mathbb{R}^{n \times p}$, forward propagation is defined by:
  - At X (layer 0)
    $$Z^{(0)} = W^{(0)} X + b^{(0)}, \quad A^{(0)} = \sigma(Z^{(0)})$$
  - At hidden layer ℓ,
    $$Z^{(\ell)} = W^{(\ell)} A^{(\ell-1)} + b^{(\ell)}, \quad A^{(\ell)} = \sigma(Z^{(\ell)})$$

Then, $\hat{Y} = A^{(L)}$, and the loss function is given by $\ell(Y, \hat{Y})$ of layer L.

Back Propagation:

$$\frac{\partial \mathcal{L}}{\partial W^{(l)}} = \frac{\partial \mathcal{L}}{\partial A^{(l)}} \cdot \frac{\partial A^{(l)}}{\partial Z^{(l)}} \cdot \frac{\partial Z^{(l)}}{\partial W^{(l)}}$$

$$= \left[\frac{\partial \mathcal{L}}{\partial A^{(l)}} \odot g'(Z^{(l)}) \right] (A^{(l-1)})^T$$

$$\frac{\partial \mathcal{L}}{\partial A^{(l)}} : \frac{\partial}{\partial A^{(l)}} \ell(\hat{y}, y) = \frac{\partial}{\partial A^{(l)}} \ell(A^{(L)}, y) = \frac{\partial}{\partial A^{(l)}} \ell\Big(g(\underbrace{W^{(l-1)} A^{(l-1)} + b^{(l-1)}}_{Z^{(l)}}), y\Big)$$

$$= \frac{\partial}{\partial A^{(l)}} \ell\Big(g(W^{(l-1)} g(\underbrace{W^{(l-2)} A^{(l-2)} + b^{(l-2)}}_{Z^{(l-1)}}) + b^{(l-1)}), y\Big)$$

...

Notes:

- $X_1, \dots, X_N$ should be standardized to prevent exploding / vanishing gradients
- One forward + backward pass is called an epoch
- With no activation function, we have that

$$\hat{y} = W^{(L-1)} W^{(L-2)} \dots W^{(1)} X + b^{(1)} + \dots + b^{(L-2)} + b^{(L-1)},$$

which is just a linear model.

Convolution

- X is the input signal
- Y is the filter / kernel that slides along X

Ex. $X = [a, b, c, d, e]$, $Y = [-1, 1]$

$X * Y = [-a+b, -b+c, -c+d, -d+e]$ — all convolutions in this class are stride 1, no padding

Ex 2.

$$X = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} \qquad Y = \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}$$

$$X * Y = \begin{bmatrix} 1 & 0 & -1 \\ 0 & 0 & 0 \\ -1 & 0 & 1 \end{bmatrix}$$

A convolution is a feed way to disseminate information throughout the network.

Ex 3.

Consider $X = [a \quad b \quad c \quad d \quad e]$

$f_1 = [f_{11} \quad f_{12}]$

$f_2 = [f_{21} \quad f_{22}]$

$$f_1 * X = [f_{11}a + f_{12}b \quad f_{11}b + f_{12}c \quad f_{11}c + f_{12}d \quad f_{11}d + f_{12}e]$$

$$f_2 * f_1 * X = [\, f_{21}(f_{11}a + f_{12}b) + f_{22}(f_{11}b + f_{12}c) \quad f_{21}(f_{11}b + f_{12}c) + f_{22}(f_{11}c + f_{12}d) \quad f_{21}(f_{11}c + f_{12}d) + f_{22}(f_{11}d + f_{12}e) \,]$$

- In each component in $f_2 * f_1 * X$, there are three data points (a, b, c), (b, c, d), (c, d, e).
- But, the data points at the edges (a, e) are represented less than data points closer to the middle.
- Each subsequent convolution reduces each dimension by 1 element
  - Given $X \in \mathbb{R}^{n \times n}$, $X * y \in \mathbb{R}^{(n-1) \times (n-1)}$

Attention

- Given $X_i \in \mathbb{R}^d$, let $q \in \mathbb{R}^d$ be an attention filter of the same dimension.

- Then, each component of q_i determines how much attention is placed on the component x_{ij}.

$$X \odot q = \begin{bmatrix} \text{---} x_1 \text{---} \\ \vdots \\ \text{---} x_n \text{---} \end{bmatrix} \begin{bmatrix} q_1 \\ \vdots \\ q_d \end{bmatrix} = \begin{bmatrix} x_{11} \cdot q_1 & x_{12} \cdot q_2 & \cdots & x_{1d} \cdot q_d \\ & \vdots & & \\ x_{n1} \cdot q_1 & x_{n2} \cdot q_2 & & x_{nd} \cdot q_d \end{bmatrix}$$

Large Language Models

- LLMs are fundamentally next-token predictors
- Each word is assigned a unique token t
- Then, given tokens $t_1 \dots t_k$, predict t_{k+1}

LLMs are trained in the following way:

1. Foundational model Training – learn weights from entire Internet
2. Mid-Training – Refine weights on questions asked on the Internet, learn a smaller set of conversational data
3. Post-training – using real people to fine-tune the model with sophisticated conversations

LLMs typically downsample user queries into a densely represented embedding vector. This embedding vector can then be compared to stored embeddings in a vector database, and can further

- Encoder/Autoencoder – reduce $x_1 \dots x_n$ to dense representation, then upsample the response.